

SQL Learning Guide

Every SQL file used to build this project's Supabase (PostgreSQL) database, with the complete code and a beginner-friendly explanation of how and why it is written.

1. SQL basics you need first	statements, data types, rules (constraints)
2. <code>schema.sql</code>	creates the 6 tables, view and security
3. <code>seed_countries.sql</code>	loads 173 countries
4. <code>reset_schema.sql</code>	safely removes everything to start over
5. <code>grant_pipeline_access.sql</code>	lets the pipeline write
6. <code>verify.sql</code>	17 automatic checks (advanced SQL)
7. The SQL behind the Python pipeline	what each API call means in SQL
8. Practice queries	safe, read-only queries to try yourself
9. Cheat sheet	one-page summary for interviews
Appendix	complete <code>seed_countries.sql</code>

The order the files were run

#	File	Why
1	<code>reset_schema.sql</code>	The first attempt left tables behind ("relation regions already exists"), so everything was removed cleanly
2	<code>schema.sql</code>	Builds the database structure
3	<code>seed_countries.sql</code>	Fills the countries table (needs regions from step 2)
4	<code>verify.sql</code>	Proves everything was built correctly (17 × PASS)
5	<code>grant_pipeline_access.sql</code>	Added later: the pipeline got "permission denied" when writing

1. SQL basics you need first

SQL (Structured Query Language) is how you talk to a relational database. Supabase runs **PostgreSQL**, one of the most popular databases, so everything here is standard PostgreSQL.

How SQL is written

- A **statement** is one instruction and ends with a semicolon `;`. A file can hold many statements.
- `-- text` is a **comment**: ignored by the database, written for humans.
- **Keywords** (`SELECT`, `create table` ...) are not case-sensitive. This project writes them in lowercase.
- **Text values** go in **single quotes**: `'East Asia'`. A quote inside text is doubled: `'Côte d'Ivoire'`.
- **Double quotes** are for *names* that contain spaces or symbols: `"public read"`.
- Spaces and line breaks don't matter — lining things up is only for readability.

The four families of SQL statements

Family	Statements	Used in this project for
DDL – Data Definition	<code>create</code> , <code>alter</code> , <code>drop</code>	Building and removing tables and views
DML – Data Manipulation	<code>select</code> , <code>insert</code> , <code>update</code> , <code>delete</code>	Reading and writing rows
DCL – Data Control	<code>grant</code> , <code>revoke</code>	Who is allowed to read or write
TCL – Transaction Control	<code>begin</code> , <code>commit</code> , <code>rollback</code>	"All or nothing" groups of changes

Data types used in this project

Type	Holds	Example column
<code>smallint</code>	Whole numbers up to 32,767	<code>region_id</code> (only 8 regions)
<code>integer</code>	Whole numbers up to ~2.1 billion	<code>articles_fetched</code>
<code>bigint</code>	Very large whole numbers	<code>article_id</code> (grows forever)
<code>text</code>	Text of any length	<code>title</code> , <code>summary</code>
<code>char(2)</code>	Text of exactly 2 characters	<code>country_code</code> ("US")
<code>boolean</code>	<code>true</code> / <code>false</code>	<code>show_on_map</code>
<code>timestampz</code>	Date + time, stored in UTC	<code>published_at</code>

Rules (constraints) — how the database protects its own data

Rule	Meaning	Example
<code>primary key</code>	Uniquely identifies each row; never empty, never repeated	<code>article_id</code>
<code>not null</code>	The column must have a value	<code>title text not null</code>
<code>unique</code>	No two rows may share this value	<code>url_normalized</code> — blocks duplicate articles
<code>check (...)</code>	Value must pass a test	<code>severity in ('low', 'medium', 'high')</code>

<code>references t (col)</code>	Foreign key: value must exist in another table	<code>region_id references regions</code>
<code>default ...</code>	Value used when none is given	<code>default now()</code>
<code>generated always as identity</code>	Auto-numbering 1, 2, 3...	every <code>..._id</code> column

Interview line

Constraints mean the database enforces the rules itself — even if the Python code has a bug, it cannot store a duplicate article, an unknown severity, or a disruption pointing to a region that doesn't exist.

2. schema.sql — building the database

Creates 6 tables, fills 2 small lists, creates 1 view, and switches on security. Run once on an empty project. Below, the file is shown section by section, exactly as written, each followed by an explanation.

The header comment

```
-- =====
-- Supply Chain Disruption Monitor – database schema
-- =====
-- What this file does:
--   1. Creates the 6 tables described in DATA_MODEL.md
--   2. Fills in the two small fixed lists (regions, disruption types)
--   3. Creates one saved query (a "view") that the feed uses
--   4. Locks the database so the public website can only READ
--
-- How to run it (once, on a new empty Supabase project):
--   Supabase dashboard → SQL Editor → New query → paste this file → Run
--
-- The country list is loaded separately (supabase/seed_countries.sql),
-- generated from the world-map file so every map_id matches a map shape.
-- =====
```

Everything starting with `--` is a comment. A good SQL file starts by saying **what it does** and **how to run it**, so anyone opening it later understands it without asking.

Section 1 — Lookup tables

```
-- -----
-- 1. LOOKUP TABLES – fixed lists that keep categories consistent
-- -----

-- Regions shown in the chart, filters and feed.
create table regions (
  region_id  smallint generated always as identity primary key, -- auto-number
  name       text      not null unique,                       -- "East Asia"
  show_on_map boolean not null default true                    -- false only for "Global"
);

-- Countries on the world map. Each belongs to exactly one region.
create table countries (
  country_code char(2) primary key, -- ISO code: "US", "CN"
  name         text  not null,       -- display name
  map_id       char(3) unique,       -- ISO numeric id used by the map file:
"840"
  region_id    smallint not null references regions (region_id)
);

-- The allowed disruption categories. The description is also given to
-- Gemini in the prompt so it classifies articles consistently.
create table disruption_types (
  disruption_type_id smallint generated always as identity primary key,
  name               text not null unique,
  description        text
);
```

Reading `create table`

The pattern is `create table name ( column type rules, column type rules, ... );` Columns are separated by commas; there is **no comma after the last column** (a very common beginner error).

Piece	What it means
<code>region_id smallint generated always as identity primary key</code>	An auto-numbered ID. <code>always</code> means you <i>cannot</i> type your own ID — the database always picks the next number. <code>primary key</code> makes it the row's unique identifier.
<code>name text not null unique</code>	Every region must have a name, and no two regions can have the same name.
<code>show_on_map boolean not null default true</code>	If an insert doesn't mention this column, it becomes <code>true</code> .
<code>country_code char(2) primary key</code>	A natural key : the ISO code itself is the ID, because it is already unique and meaningful ("US").
<code>region_id smallint not null references regions (region_id)</code>	A foreign key . You cannot add a country with <code>region_id = 99</code> if region 99 doesn't exist, and you cannot delete a region that countries still use.

Why lookup tables?

If the AI wrote 'E. Asia' one day and 'East Asia' the next, a filter on region would silently miss rows. With a lookup table, every row stores the region's ID, so the name is spelled one way, in one place.

Section 2 — The pipeline log

```
-----
-- 2. PIPELINE LOG — one row every time the daily job runs
-----

create table pipeline_runs (
  run_id          bigint generated always as identity primary key,
  started_at      timestampz not null default now(),
  finished_at     timestampz,                -- empty while still running
  status          text not null default 'running'
               check (status in ('running', 'success', 'failed')),
  articles_fetched integer,                  -- returned by the news API
  articles_new    integer,                  -- left after removing duplicates
  disruptions_found integer,               -- flagged by Gemini as real
  error_message   text                    -- what went wrong, if anything
);
```

- `default now()` — when a row is inserted without `started_at`, the database fills in the current time.
- `finished_at timestampz`, has **no not null**, so it may be empty (`null`) — it is empty while the run is still going.
- `check (status in ('running', 'success', 'failed'))` — only these three words are accepted. Anything else is rejected with an error.

Section 3 — Articles

```
-----
-- 3. ARTICLES — every unique article fetched (real disruption or not)
-----

create table articles (
  article_id      bigint generated always as identity primary key,
```

```

run_id          bigint not null references pipeline_runs (run_id),
url             text  not null,                -- original link for "Open source article"
url_normalized  text  not null unique,         -- cleaned URL; UNIQUE blocks duplicates
title           text  not null,                -- headline on the feed card
title_normalized text not null,                -- cleaned headline for the 7-day duplicate check
source_name     text,                          -- "Reuters"
description     text,                          -- short snippet; title + snippet go to Gemini
search_keyword  text,                          -- which keyword found it
published_at    timestampz not null,           -- publisher's date (used by charts and cards)
fetched_at      timestampz not null default now(),
status          text not null default 'pending'
               check (status in ('pending', 'disruption', 'not_disruption', 'failed'))
);

```

- `run_id bigint not null references pipeline_runs (run_id)` — every article remembers which run found it; that run must exist.
- `url_normalized text not null unique` — **the duplicate blocker**. If the pipeline tries to save the same (cleaned) URL twice, the database refuses.
- `status ... default 'pending' + check` — new articles start as *pending* (waiting for Gemini) and can only ever hold the four listed values.

Section 4 — Disruptions

```

-----
-- 4. DISRUPTIONS - Gemini's structured result, only for real disruptions
-----

create table disruptions (
  disruption_id  bigint generated always as identity primary key,
  article_id     bigint not null unique                -- one result per article
                  references articles (article_id) on delete cascade,
  region_id      smallint not null references regions (region_id),
  country_code   char(2) references countries (country_code), -- optional
  disruption_type_id smallint not null references disruption_types (disruption_type_id),
  severity        text not null
                  check (severity in ('low', 'medium', 'high')),
  summary         text not null,                      -- one plain-English sentence
  model_name      text not null,                      -- which Gemini model produced it
  extracted_at    timestampz not null default now()
);

```

- `article_id bigint not null unique references articles (article_id) on delete cascade` — three rules in one line:
 - `references`: must point to a real article;
 - `unique`: an article can have **at most one** AI result (a *one-to-one* relationship);
 - `on delete cascade`: if the article is deleted, its disruption row is deleted automatically, so no "orphan" rows are left.
- `country_code char(2) references countries (country_code)` — no `not null`, so it is **optional** (some disruptions, like "Red Sea shipping", have no single country). But if it's filled in, it must be a real country.
- `check (severity in ('low', 'medium', 'high'))` — this is what keeps the red/yellow/green colours consistent everywhere.

Section 5 — Starting data

```
-- 5. STARTING DATA for the fixed lists
```

```
insert into regions (name, show_on_map) values
  ('East Asia',      true),
  ('South Asia',     true),
  ('Middle East',    true),
  ('Europe',         true),
  ('North America',  true),
  ('Latin America',  true),
  ('Africa & Oceania', true),
  ('Global',         false); -- worldwide events; no single place on the map

insert into disruption_types (name, description) values
  ('Labour action',   'Strikes, walkouts, lockouts or labour disputes that stop or slow work.'),
  ('Weather event',   'Storms, typhoons, floods, drought, low water levels or other natural
events.'),
  ('Tariff / policy', 'New tariffs, export controls, sanctions, customs rules or trade
regulations.'),
  ('Geopolitical',    'Conflict, security threats or political instability affecting trade
routes.'),
  ('Supplier shortage', 'A supplier or input material cannot meet demand.'),
  ('Port / shipping delay', 'Port congestion, vessel delays, canal restrictions or freight backlogs.'),
  ('Other',           'A real supply chain disruption that fits none of the categories above.');
```

The pattern is `insert into table (columns) values (row 1), (row 2), ...;`

- You list the columns you are filling, then one bracket per row, in the same order.
- `region_id` is not listed — it is auto-numbered, so the regions get IDs 1–8 in the order written.
- One `insert` with many rows is faster and tidier than eight separate inserts.

Section 6 — The feed view

```
-- -----
-- 6. FEED VIEW – joins everything the feed card needs into one place,
--   so the website makes one simple request instead of five joins.
--   security_invoker = the view follows the same read rules as the tables.
-- -----
```

```
create view disruption_feed
with (security_invoker = true) as
select
  d.disruption_id,
  d.severity,
  d.summary,
  d.model_name,
  d.extracted_at,
  a.title,
  a.url,
  a.source_name,
  a.published_at,
  r.region_id,
  r.name as region,
  c.country_code,
  c.name as country,
  t.disruption_type_id,
  t.name as disruption_type
from disruptions d
```

```

join articles      a on a.article_id      = d.article_id
join regions       r on r.region_id       = d.region_id
join disruption_types t on t.disruption_type_id = d.disruption_type_id
left join countries c on c.country_code    = d.country_code; -- left join: country is optional

```

A **view** is a **saved query** that behaves like a read-only table. The website can simply ask for `disruption_feed` instead of repeating this join every time.

Piece	What it means
<code>from disruptions d</code>	<code>d</code> is an alias — a short nickname, so we can write <code>d.severity</code> instead of <code>disruptions.severity</code> .
<code>join articles a on a.article_id = d.article_id</code>	Join = combine rows from two tables where the condition matches. Each disruption is paired with its article.
<code>left join countries c ...</code>	A normal (inner) join would drop disruptions with no country. <code>left join</code> keeps every disruption and leaves the country columns empty when there's no match.
<code>r.name as region</code>	Renames the column in the result (both tables have a column called <code>name</code>).
<code>with (security_invoker = true)</code>	The view obeys the same security rules as the tables it reads — it can't be used as a back door.

Section 7 — Security (Row Level Security)

```

-----
-- 7. SECURITY — Row Level Security (RLS)
-- The website uses the public "anon" key → may only SELECT (read).
-- The pipeline uses the secret "service role" key → bypasses RLS and
-- is the only thing that can insert or update.
-----

alter table regions      enable row level security;
alter table countries    enable row level security;
alter table disruption_types enable row level security;
alter table pipeline_runs enable row level security;
alter table articles     enable row level security;
alter table disruptions   enable row level security;

create policy "public read" on regions      for select to anon, authenticated using (true);
create policy "public read" on countries    for select to anon, authenticated using (true);
create policy "public read" on disruption_types for select to anon, authenticated using (true);
create policy "public read" on pipeline_runs for select to anon, authenticated using (true);
create policy "public read" on articles     for select to anon, authenticated using (true);
create policy "public read" on disruptions   for select to anon, authenticated using (true);

-- Make sure the website's key is allowed to read these tables and the view
-- (read only – no insert/update/delete is granted).
grant select on regions, countries, disruption_types, pipeline_runs,
              articles, disruptions, disruption_feed
to anon, authenticated;

```

PostgreSQL has **two layers** of protection, and a request must pass both:

1. **GRANT** — "may this role touch this table at all, and for what (read / insert / update / delete)?"
 2. **Row Level Security (RLS)** — "which rows may this role see or change?"
- `alter table ... enable row level security` — from now on, a role sees **no rows** unless a policy allows it.
 - `create policy "public read" on regions for select to anon, authenticated using (true)` — for reading (`select`), the website's roles may see rows where the condition is true; `using (true)` means every row. There

is no policy for insert/update/delete, so those are blocked.

- `grant select on ... to anon, authenticated` — the table-level permission to read.
- **anon** = a visitor using the public key; **authenticated** = a logged-in user (not used yet).

Lesson learned

This section gave the website read access but never granted anything to `service_role` (the pipeline's role). The secret key skips RLS, but it still needs GRANTS — newer Supabase projects don't add them automatically. That's why `grant_pipeline_access.sql` (section 5) was needed later.

Interview line

The website's key can only read — even if someone copies it out of the page, they can't change data. Only the pipeline, holding a secret key on the server, can write.

3. seed_countries.sql — loading 173 countries

"Seed" data is the starting data an app needs. This file was **generated** by a Python script from the world-map file and the official ISO code list, so no code was typed by hand. The shortened version below shows the pattern (the full file is in the Appendix).

seed_countries.sql (shortened)

```
-- =====
-- Country list for the world map (GENERATED - do not edit by hand)
-- Made by supabase/generate_seed_countries.py from:
--   map shapes : https://cdn.jsdelivr.net/npm/world-atlas@2.0.2/countries-110m.json
--   ISO codes  : https://cdn.jsdelivr.net/gh/Lukes/ISO-3166-Countries-with-Regional-
Codes@10.0/all/all.json
-- Run AFTER schema.sql (it needs the regions table to be filled).
--
-- 173 countries. Not included:
--   no ISO code on the map (stay grey): Kosovo, N. Cyprus, Somaliland
--   skipped: Antarctica
-- =====

insert into countries (country_code, name, map_id, region_id)
select v.country_code, v.name, v.map_id, r.region_id
from (values
  -- East Asia (16)
  ('BN', 'Brunei', '096', 'East Asia'),
  ('KH', 'Cambodia', '116', 'East Asia'),
  ('CN', 'China', '156', 'East Asia'),
  ('ID', 'Indonesia', '360', 'East Asia'),
  ('JP', 'Japan', '392', 'East Asia'),
  ('LA', 'Laos', '418', 'East Asia'),
  -- ... 160 more rows, same pattern ...
  ('SZ', 'eSwatini', '748', 'Africa & Oceania')
) as v (country_code, name, map_id, region_name)
join regions r on r.name = v.region_name;

-- Check: the join above silently drops a row whose region name has no match,
-- so this should return 173:
-- select count(*) from countries;
```

How this statement works, step by step

1. `from (values (...), (...)) as v (country_code, name, map_id, region_name)`
`values` builds a small **temporary table** right inside the query. `as v (...)` names it `v` and names its four columns.
2. `join regions r on r.name = v.region_name`
Each row names its region in words ("East Asia"), but the `countries` table needs the region's *number*. The join looks up each name in `regions` and finds its `region_id`.
3. `select v.country_code, v.name, v.map_id, r.region_id`
Picks the four values to store: three from the temporary table, the ID from `regions`.
4. `insert into countries (...) + select`
Instead of `values`, an insert can take its rows from a `select`. Every row the select produces is inserted.

Why not just write the region numbers?

Numbers like 7 mean nothing when you read the file, and would silently be wrong if the regions were ever inserted in a different order. Writing names and letting the join translate them is readable and safe.

The one trap — and the check for it

An (inner) join drops rows that have no match. A misspelled region name ('East Aisa') would make that country silently disappear. That's why the file ends with a reminder: `select count(*) from countries;` must return 173 — and `verify.sql` checks it automatically.

Other details: `'Côte d'Ivoire'` shows the doubled-quote rule; `map_id` values like `'004'` are stored as text so the leading zeros are kept, matching the map file exactly.

4. reset_schema.sql — removing everything safely

Written after the first run failed with "relation regions already exists". It removes only this project's objects so `schema.sql` can run on a clean database. **It deletes data** — only for use before real data exists.

```
-- =====
-- RESET — removes everything schema.sql and seed_countries.sql created,
-- so schema.sql can be run again on a clean project.
--
-- ⚠ Deletes all data in these tables. Only use before real data exists.
--
-- Safe by design:
-- • Names only this project's 1 view + 6 tables — nothing else is touched.
-- • "if exists" = no error if an object was never created (partial run).
-- • No CASCADE: if anything unexpected depends on these tables, Postgres
--   stops with an error instead of silently deleting it.
-- • Wrapped in begin/commit: either everything is removed or nothing is.
--
-- Dropping a table automatically removes what belongs to it: its data,
-- RLS policies, keys, unique/check rules, auto-number sequences and grants.
-- =====

begin;

-- The view reads from the tables, so it goes first.
drop view if exists public.disruption_feed;

-- Tables in reverse dependency order: a table is dropped only after
-- every table that points to it (via a foreign key) is already gone.
drop table if exists public.disruptions;      -- points to articles, regions, countries, types
drop table if exists public.articles;         -- points to pipeline_runs
drop table if exists public.pipeline_runs;
drop table if exists public.countries;        -- points to regions
drop table if exists public.disruption_types;
drop table if exists public.regions;

commit;

-- Check: should return no rows (nothing from this project is left).
select c.relname as leftover_object, c.relkind as kind
from pg_class c
join pg_namespace n on n.oid = c.relnamespace
where n.nspname = 'public'
and c.relname in ('disruption_feed', 'disruptions', 'articles', 'pipeline_runs',
                  'countries', 'disruption_types', 'regions');
```

Explanation

Piece	What it means
<code>begin; ... commit;</code>	A transaction : everything in between happens together or not at all . If any drop failed, none of them would take effect — the database is never left half-deleted.
<code>drop view if exists public.disruption_feed;</code>	<code>drop</code> deletes an object. <code>if exists</code> = no error if it isn't there (useful after a partial run). <code>public.</code> is the schema (folder) the object lives in.
The order of the drops	A table can't be dropped while another table's foreign key points to it. So children go first (<code>disruptions</code> → <code>articles</code> → ...) and <code>regions</code> , which everything points to, goes last. The view goes first because it reads from the tables.

No <code>cascade</code>	<code>drop ... cascade</code> would also delete anything depending on the table. Leaving it out means an unexpected dependency causes an error instead of silently deleting something.
The final <code>select</code>	Reads <code>pg_class</code> , PostgreSQL's own list of every table and view (the system catalog). Zero rows = nothing from the project is left. <code>relkind</code> is <code>r</code> for a table, <code>v</code> for a view.

Dropping a table removes everything that belongs to it

Its rows, RLS policies, keys, check rules, auto-number counter and grants all go with it — which is why the file only needs to name the view and the six tables.

5. grant_pipeline_access.sql — letting the pipeline write

Added when the first real pipeline run failed with `permission denied for table pipeline_runs` (error code 42501).

```
-- =====
-- Give the pipeline permission to write – run once, after schema.sql.
--
-- Why this is needed:
--   The pipeline connects with the SECRET key, which acts as the
--   "service_role" database role. That role skips Row Level Security,
--   but it still needs ordinary table permissions (GRANTS). Newer Supabase
--   projects no longer add these automatically for new tables, and
--   schema.sql only granted READ access to the website's roles.
--
-- Error this fixes:
--   permission denied for table pipeline_runs (code 42501)
--
-- The public website's roles (anon, authenticated) are NOT changed:
-- they stay read-only.
-- =====

grant select, insert, update, delete
  on regions, countries, disruption_types, pipeline_runs, articles, disruptions
  to service_role;

grant select on disruption_feed to service_role;

-- The auto-numbering counters behind the ID columns.
grant usage, select on all sequences in schema public to service_role;

-- Check: every row should say true.
select t as table_name,
       has_table_privilege('service_role', 'public.' || t, 'INSERT') as pipeline_can_insert,
       has_table_privilege('service_role', 'public.' || t, 'UPDATE') as pipeline_can_update,
       not has_table_privilege('anon', 'public.' || t, 'INSERT') as website_still_read_only
from unnest(array['regions', 'countries', 'disruption_types',
                  'pipeline_runs', 'articles', 'disruptions']) as t;
```

Explanation

- `grant select, insert, update, delete on ... to service_role;` — the pattern is `grant <actions> on <tables> to <role>`. `service_role` is the database role used by the secret key.
- `grant usage, select on all sequences in schema public` — a **sequence** is the counter behind each auto-numbered ID column. `all ... in schema public` covers every one at once.
- The website's roles (`anon`, `authenticated`) are not mentioned, so they stay read-only.

The check query

- `unnest(array['regions', ...]) as t` — `array[...]` is a list; `unnest` turns the list into rows (one row per table name), so one query can check all six tables.
- `has_table_privilege('service_role', 'public.' || t, 'INSERT')` — a built-in function that answers true/false: "may this role do this action on this table?" `||` joins text together (`'public.' || 'articles'` → `'public.articles'`).
- `not has_table_privilege('anon', ...)` — `not` flips true/false, so `true` here means "the website still can't insert" — the result we want.

Interview line

Postgres security has two layers: GRANTS decide which tables a role may touch, Row Level Security decides which rows. The secret key skips RLS but still needed GRANTS — I diagnosed that from the error code and fixed it with a targeted grant, without widening the website's access.

6. verify.sql — 17 automatic checks

The most advanced file. Instead of clicking through Supabase to check everything, one query tests the tables, relationships, rules, security, view and seed counts, and prints PASS or FAIL for each.

```
-- =====
-- Verification checks – run AFTER schema.sql and seed_countries.sql.
-- Read-only except for one write test that is expected to be BLOCKED.
-- Output: one table of checks, each marked PASS or FAIL.
-- =====
```

Part 1 — A real security test: pretending to be the website

```
-- ---- Security test: act as the public website ("anon") ----
-- Try to READ (should work) and WRITE (should be blocked by RLS).
drop table if exists _anon_test;
create temp table _anon_test (check_name text, actual text);

do $$
declare n integer;
begin
    -- read test
    begin
        set local role anon;
        select count(*) into n from public.regions;
        reset role;
        insert into _anon_test values ('anon can read regions', n::text);
    exception when others then
        insert into _anon_test values ('anon can read regions', 'error: ' || sqlerrm);
    end;

    -- write test
    begin
        set local role anon;
        insert into public.regions (name) values ('__rls_test__');
        reset role;
        insert into _anon_test values ('anon write is blocked', 'NOT blocked');
    exception when insufficient_privilege then
        insert into _anon_test values ('anon write is blocked', 'blocked');
    end;
end $$;

-- If the write was NOT blocked, remove the test row again.
delete from public.regions where name = '__rls_test__';
```

Piece	What it means
create temp table _anon_test (...)	A temporary table : exists only during this session and disappears afterwards. Used as a notepad to hold the test results. <code>drop table if exists</code> first makes the file safe to run twice.
do \$\$... \$\$;	An anonymous code block in PL/pgSQL, PostgreSQL's programming language. Plain SQL has no "try this, and if it fails do that" — PL/pgSQL does. <code>\$\$</code> marks the start and end of the code.
declare n integer;	Creates a variable <code>n</code> to hold a number.

<code>begin ... exception when ... then ... end;</code>	Error handling , like try/except in Python. If anything between <code>begin</code> and <code>exception</code> fails, the matching <code>when</code> branch runs instead, and all changes in that block are undone.
<code>set local role anon;</code>	Switch identity to the website's role for the rest of this block. Everything after this runs with the website's permissions.
<code>select count(*) into n from public.regions;</code>	Count rows as the website would, and store the number in <code>n</code> . Expected: 8.
<code>reset role;</code>	Switch back to the administrator, so the result can be written into the temp table.
<code>insert into public.regions (name) values ('__rls_test__');</code>	Try to write as the website. This <i>should</i> fail.
<code>exception when insufficient_privilege</code>	Catches exactly the "not allowed" error (code 42501). Reaching this branch means security worked , so it records <code>'blocked'</code> .
<code>sqlerrm</code>	The text of the error message, if an unexpected error happens.
<code>delete from public.regions where name = '__rls_test__';</code>	Clean-up: if the write had <i>not</i> been blocked, remove the test row. (This is why Supabase warns about a "destructive operation".)

Part 2 — All checks in one query

```
-- ---- ALL checks -----
with our_tables(t) as (
  values ('regions'), ('countries'), ('disruption_types'),
         ('pipeline_runs'), ('articles'), ('disruptions')
),
checks(n, check_name, expected, actual) as (

  select 1, 'tables created', '6',
         (select count(*) from pg_tables
          where schemaname = 'public' and tablename in (select t from our_tables))::text

  union all select 2, 'RLS enabled on all tables', '6',
         (select count(*) from pg_class c join pg_namespace s on s.oid = c.relnamespace
          where s.nspname = 'public' and c.relname in (select t from our_tables)
          and c.relrowsecurity)::text

  union all select 3, 'read-only policies', '6',
         (select count(*) from pg_policies
          where schemaname = 'public' and cmd = 'SELECT')::text

  union all select 4, 'policies allowing writes', '0',
         (select count(*) from pg_policies
          where schemaname = 'public' and cmd <> 'SELECT')::text

  union all select 5, 'foreign keys (relationships)', '6',
         (select count(*) from pg_constraint
          where contype = 'f' and connamespace = 'public'::regnamespace)::text

  union all select 6, 'unique rules', '5',
         (select count(*) from pg_constraint
          where contype = 'u' and connamespace = 'public'::regnamespace)::text

  union all select 7, 'allowed-value rules (severity, statuses)', '3',
         (select count(*) from pg_constraint
          where contype = 'c' and connamespace = 'public'::regnamespace)::text
```

```

union all select 8, 'feed view exists and follows RLS', 'true',
  (select coalesce(array_to_string(reloptions, ','), '') like '%security_invoker=true%'
   from pg_class where oid = 'public.disruption_feed'::regclass)::text

union all select 9, 'feed view is readable by website', 'true',
  has_table_privilege('anon', 'public.disruption_feed', 'SELECT')::text

union all select 10, 'regions seeded', '8',
  (select count(*) from regions)::text

union all select 11, 'disruption types seeded', '7',
  (select count(*) from disruption_types)::text

union all select 12, 'countries seeded', '173',
  (select count(*) from countries)::text

union all select 13, 'countries per region',
  'East Asia 16, South Asia 12, Middle East 19, Europe 38, North America 4, Latin America 27,
  Africa & Oceania 57, Global 0',
  (select string_agg(r.name || ' ' || (select count(*) from countries c
                                     where c.region_id = r.region_id), ', '
                    order by r.region_id)
   from regions r)

union all select 14, 'spot check: US → North America, map id 840', 'North America 840',
  (select r.name || ' ' || c.map_id from countries c join regions r using (region_id)
   where c.country_code = 'US')

union all select 15, 'data tables start empty (runs, articles, disruptions)', '0 0 0',
  (select (select count(*) from pipeline_runs) || ' ' ||
   (select count(*) from articles) || ' ' ||
   (select count(*) from disruptions))

union all select 16, 'anon can read regions', '8',
  (select actual from _anon_test where check_name = 'anon can read regions')

union all select 17, 'anon write is blocked', 'blocked',
  (select actual from _anon_test where check_name = 'anon write is blocked')
)
select n as "#",
  check_name,
  expected,
  actual,
  case when expected = actual then 'PASS' else 'FAIL' end as result
from checks
order by n;

```

New ideas in this query

Piece	What it means
<code>with our_tables(t) as (values ...), checks(...) as (...)</code>	A CTE (Common Table Expression): a named temporary result you can use later in the same query. Makes long queries readable, like variables.
<code>select 1, 'tables created', '6', (...)</code>	Each check is one row: number, name, expected value, actual value.
<code>union all</code>	Stacks the results of several <code>select</code> s into one list of rows (<code>union</code> without <code>all</code> would also remove duplicate rows).
<code>(select count(*) from ...)</code>	A subquery : a query inside another query, used here to calculate each actual value.

<code>...::text</code>	A cast : convert a value to another type. Counts are numbers; turning them into text lets every row be compared the same way.
<code>pg_tables, pg_class, pg_policies, pg_constraint</code>	PostgreSQL's system catalog — tables describing the database itself. <code>relrowsecurity</code> = is RLS on? <code>cmd</code> = which action a policy covers.
<code>contype = 'f' / 'u' / 'c'</code>	Constraint type: f oreign key, u nique, c heck. Expected: 6, 5 and 3.
<code>'public'::regnamespace, 'public.disruption_feed'::regclass</code>	Special casts that turn a name into the internal ID PostgreSQL uses, so it can be compared with catalog columns.
<code>string_agg(..., ' ', ' order by r.region_id)</code>	An aggregate that glues many rows into one text value: "East Asia 16, South Asia 12, ...".
<code>(select count(*) from countries c where c.region_id = r.region_id)</code>	A correlated subquery : runs once per region, using the current region's ID.
<code>join regions r using (region_id)</code>	Short form of <code>on c.region_id = r.region_id</code> when both columns have the same name.
<code>case when expected = actual then 'PASS' else 'FAIL' end</code>	SQL's if/else : produces a value depending on a condition.

Interview line

I wrote a verification script that tests the database the way the website would actually use it — it logs in as the public role, proves reads work and writes are blocked, and checks every table, relationship and seed count in one run.

7. The SQL behind the Python pipeline

The Python code never writes SQL text. Supabase gives every table a web address (a REST API, run by a service called PostgREST) and turns each HTTP request into SQL. This table shows the SQL each request becomes — reading it is a great way to connect the two.

What the pipeline does	HTTP request (from db.py)	Equivalent SQL
Start a run	POST /pipeline_runs {status: running} Prefer: return=representation	<pre>insert into pipeline_runs (status) values ('running') returning run_id;</pre>
Finish a run	PATCH /pipeline_runs?run_id=eq.1	<pre>update pipeline_runs set status = 'success', finished_at = now(), articles_fetched = 31, articles_new = 27, disruptions_found = 0, error_message = null where run_id = 1;</pre>
Is this URL already stored?	GET /articles?select=url_normalized &url_normalized=in("...", "...")	<pre>select url_normalized from articles where url_normalized in ('https://a.com/x', 'https://b.com/y');</pre>
Titles from the last 7 days	GET /articles?select=title_normalized &published_at=gte.<date>&limit=1000	<pre>select title_normalized from articles where published_at >= now() - interval '7 days' order by article_id limit 1000 offset 0;</pre>
Save new articles	POST /articles?on_conflict=url_normalized Prefer: resolution=ignore-duplicates	<pre>insert into articles (run_id, url, url_normalized, title, ...) values (1, 'https://...', 'https://...', 'Port strike...', ...), (1, ...) on conflict (url_normalized) do nothing returning article_id;</pre>
Articles waiting for Gemini	GET /articles?status=in.(pending,failed) &order=status.desc,article_id.asc&limit=50	<pre>select article_id, title, description, source_name, published_at from articles where status in ('pending', 'failed') order by status desc, article_id asc limit 50;</pre>
Save a disruption	POST /disruptions?on_conflict=article_id Prefer: resolution=ignore-duplicates	<pre>insert into disruptions (article_id, region_id, country_code, disruption_type_id, severity, summary, model_name) values (1, 5, 'US', 7, 'low', 'A key holiday supplier...', 'gemini-3.5-flash') on conflict (article_id) do nothing;</pre>

Record Gemini's verdict	PATCH /articles?article_id=eq.1 {status: disruption}	<pre>update articles set status = 'disruption' where article_id = 1;</pre>
-------------------------	--	--

Two ideas worth remembering

- **returning** — makes an `insert` or `update` send back the rows it changed. The pipeline uses it to learn the new `run_id`, and to count how many articles were *really* inserted.
- **on conflict (...) do nothing** — "if this would break a unique rule, skip this row quietly instead of failing". It is the pipeline's safety net against duplicates.
- **URL filters map to where:** `eq.` is `=`, `gte.` is `>=`, `in(...)` is `in (...)`.

8. Practice queries (safe — they only read)

Paste any of these into Supabase → SQL Editor → New query. None of them change data. Several are exactly the calculations the dashboard will need.

1. How many articles are in each status?

```
select status, count(*) as articles
from articles
group by status
order by articles desc;
```

`group by` puts rows into groups (one per status) and `count(*)` counts each group.

2. The last 5 pipeline runs

```
select run_id, status, articles_fetched, articles_new,
       disruptions_found, started_at, finished_at
from pipeline_runs
order by run_id desc
limit 5;
```

`order by ... desc` = newest first; `limit 5` = only 5 rows.

3. The live feed (what the website will show)

```
select published_at, severity, region, country, disruption_type, title, summary
from disruption_feed
order by published_at desc
limit 10;
```

Reads the view like a normal table — all the joins are already done.

4. Dashboard KPI cards (last 30 days)

```
select
  (select count(*) from articles
   where status in ('disruption', 'not_disruption')
   and published_at >= now() - interval '30 days') as articles_screened,
  (select count(*) from disruption_feed
   where published_at >= now() - interval '30 days') as disruption_signals,
  (select count(*) from disruption_feed
   where severity = 'high'
   and published_at >= now() - interval '30 days') as high_severity;
```

Three subqueries side by side give one row with three numbers. `now() - interval '30 days'` = 30 days ago.

5. World map: highest severity per country

```
select country,
       count(*) as signals,
       case max(case severity when 'high' then 3
                        when 'medium' then 2
                        else 1 end)
          when 3 then 'high' when 2 then 'medium' else 'low'
       end as highest_severity
from disruption_feed
where country_code is not null
      and published_at >= now() - interval '30 days'
```

```
group by country
order by signals desc;
```

Text can't be ranked, so the inner `case` turns severity into 1–3, `max` takes the highest, and the outer `case` turns it back into a word. `is not null` skips disruptions with no country.

6. Trend chart: signals per week per region

```
select date_trunc('week', published_at) as week,
       region,
       count(*) as signals
from disruption_feed
where published_at >= now() - interval '8 weeks'
group by week, region
order by week, region;
```

`date_trunc('week', ...)` rounds each date down to the Monday of its week, so articles group by week.

7. Which search keywords find real disruptions?

```
select search_keyword,
       count(*) as articles,
       count(*) filter (where status = 'disruption') as disruptions
from articles
group by search_keyword
order by disruptions desc;
```

`filter (where ...)` counts only matching rows inside a group. Good material for the README's findings.

Don't practise with these

`reset_schema.sql` (deletes everything), and any `insert`, `update`, `delete` or `drop` on the real tables.

9. Cheat sheet

You want to...	SQL
Create a table	<code>create table t (id bigint generated always as identity primary key, name text not null);</code>
Add rows	<code>insert into t (name) values ('a'), ('b');</code>
Add rows, skip duplicates	<code>insert ... on conflict (col) do nothing;</code>
Read rows	<code>select col1, col2 from t where ... order by ... limit 10;</code>
Count per group	<code>select col, count(*) from t group by col;</code>
Combine tables	<code>from a join b on b.id = a.b_id (keep unmatched rows of a: left join)</code>
Change rows	<code>update t set col = 'x' where id = 1; — always include where !</code>
Remove rows	<code>delete from t where id = 1; — always include where !</code>
Save a query	<code>create view v as select ...;</code>
All-or-nothing changes	<code>begin; ... commit;</code>
Give permission	<code>grant select on t to role;</code>
Row-level access	<code>alter table t enable row level security; + create policy ...</code>
If / else	<code>case when x then 'a' else 'b' end</code>
Dates	<code>now(), now() - interval '7 days', date_trunc('week', col)</code>

How to explain this database in an interview (60 seconds)

1. "There are six tables. **articles** stores what the news said, **disruptions** stores what the AI concluded, **regions / countries / disruption_types** are lookup lists that keep categories consistent, and **pipeline_runs** logs every daily run."
2. "The database enforces its own rules: a unique rule blocks duplicate articles, check rules allow only valid severities and statuses, and foreign keys stop rows pointing at things that don't exist."
3. "A view joins everything the feed needs, so the website makes one simple request."
4. "Security has two layers: GRANTS and Row Level Security. The public website can only read; only the pipeline's secret key can write."
5. "The country list was generated from the same map file the dashboard draws, and a verification script proves everything — including the security — with 17 automatic checks."

Appendix — complete seed_countries.sql

Exactly as in the project folder (generated by `supabase/generate_seed_countries.py`).

```
-- =====
-- Country list for the world map (GENERATED - do not edit by hand)
-- Made by supabase/generate_seed_countries.py from:
--   map shapes : https://cdn.jsdelivr.net/npm/world-atlas@2.0.2/countries-110m.json
--   ISO codes  : https://cdn.jsdelivr.net/gh/Lukes/ISO-3166-Countries-with-Regional-
Codes@10.0/all/all.json
-- Run AFTER schema.sql (it needs the regions table to be filled).
--
-- 173 countries. Not included:
--   no ISO code on the map (stay grey): Kosovo, N. Cyprus, Somaliland
--   skipped: Antarctica
-- =====

insert into countries (country_code, name, map_id, region_id)
select v.country_code, v.name, v.map_id, r.region_id
from (values
  -- East Asia (16)
  ('BN', 'Brunei', '096', 'East Asia'),
  ('KH', 'Cambodia', '116', 'East Asia'),
  ('CN', 'China', '156', 'East Asia'),
  ('ID', 'Indonesia', '360', 'East Asia'),
  ('JP', 'Japan', '392', 'East Asia'),
  ('LA', 'Laos', '418', 'East Asia'),
  ('MY', 'Malaysia', '458', 'East Asia'),
  ('MN', 'Mongolia', '496', 'East Asia'),
  ('MM', 'Myanmar', '104', 'East Asia'),
  ('KP', 'North Korea', '408', 'East Asia'),
  ('PH', 'Philippines', '608', 'East Asia'),
  ('KR', 'South Korea', '410', 'East Asia'),
  ('TW', 'Taiwan', '158', 'East Asia'),
  ('TH', 'Thailand', '764', 'East Asia'),
  ('TL', 'Timor-Leste', '626', 'East Asia'),
  ('VN', 'Vietnam', '704', 'East Asia'),
  -- South Asia (12)
  ('AF', 'Afghanistan', '004', 'South Asia'),
  ('BD', 'Bangladesh', '050', 'South Asia'),
  ('BT', 'Bhutan', '064', 'South Asia'),
  ('IN', 'India', '356', 'South Asia'),
  ('KZ', 'Kazakhstan', '398', 'South Asia'),
  ('KG', 'Kyrgyzstan', '417', 'South Asia'),
  ('NP', 'Nepal', '524', 'South Asia'),
  ('PK', 'Pakistan', '586', 'South Asia'),
  ('LK', 'Sri Lanka', '144', 'South Asia'),
  ('TJ', 'Tajikistan', '762', 'South Asia'),
  ('TM', 'Turkmenistan', '795', 'South Asia'),
  ('UZ', 'Uzbekistan', '860', 'South Asia'),
  -- Middle East (19)
  ('AM', 'Armenia', '051', 'Middle East'),
  ('AZ', 'Azerbaijan', '031', 'Middle East'),
  ('CY', 'Cyprus', '196', 'Middle East'),
  ('EG', 'Egypt', '818', 'Middle East'),
  ('GE', 'Georgia', '268', 'Middle East'),
  ('IR', 'Iran', '364', 'Middle East'),
  ('IQ', 'Iraq', '368', 'Middle East'),
  ('IL', 'Israel', '376', 'Middle East'),
  ('JO', 'Jordan', '400', 'Middle East'),
  ('KW', 'Kuwait', '414', 'Middle East'),
  ('LB', 'Lebanon', '422', 'Middle East'),
  ('OM', 'Oman', '512', 'Middle East'),
```

```

('PS', 'Palestine', '275', 'Middle East'),
('QA', 'Qatar', '634', 'Middle East'),
('SA', 'Saudi Arabia', '682', 'Middle East'),
('SY', 'Syria', '760', 'Middle East'),
('TR', 'Turkey', '792', 'Middle East'),
('AE', 'United Arab Emirates', '784', 'Middle East'),
('YE', 'Yemen', '887', 'Middle East'),
-- Europe (38)
('AL', 'Albania', '008', 'Europe'),
('AT', 'Austria', '040', 'Europe'),
('BY', 'Belarus', '112', 'Europe'),
('BE', 'Belgium', '056', 'Europe'),
('BA', 'Bosnia and Herz.', '070', 'Europe'),
('BG', 'Bulgaria', '100', 'Europe'),
('HR', 'Croatia', '191', 'Europe'),
('CZ', 'Czechia', '203', 'Europe'),
('DK', 'Denmark', '208', 'Europe'),
('EE', 'Estonia', '233', 'Europe'),
('FI', 'Finland', '246', 'Europe'),
('FR', 'France', '250', 'Europe'),
('DE', 'Germany', '276', 'Europe'),
('GR', 'Greece', '300', 'Europe'),
('HU', 'Hungary', '348', 'Europe'),
('IS', 'Iceland', '352', 'Europe'),
('IE', 'Ireland', '372', 'Europe'),
('IT', 'Italy', '380', 'Europe'),
('LV', 'Latvia', '428', 'Europe'),
('LT', 'Lithuania', '440', 'Europe'),
('LU', 'Luxembourg', '442', 'Europe'),
('MK', 'Macedonia', '807', 'Europe'),
('MD', 'Moldova', '498', 'Europe'),
('ME', 'Montenegro', '499', 'Europe'),
('NL', 'Netherlands', '528', 'Europe'),
('NO', 'Norway', '578', 'Europe'),
('PL', 'Poland', '616', 'Europe'),
('PT', 'Portugal', '620', 'Europe'),
('RO', 'Romania', '642', 'Europe'),
('RU', 'Russia', '643', 'Europe'),
('RS', 'Serbia', '688', 'Europe'),
('SK', 'Slovakia', '703', 'Europe'),
('SI', 'Slovenia', '705', 'Europe'),
('ES', 'Spain', '724', 'Europe'),
('SE', 'Sweden', '752', 'Europe'),
('CH', 'Switzerland', '756', 'Europe'),
('UA', 'Ukraine', '804', 'Europe'),
('GB', 'United Kingdom', '826', 'Europe'),
-- North America (4)
('CA', 'Canada', '124', 'North America'),
('GL', 'Greenland', '304', 'North America'),
('MX', 'Mexico', '484', 'North America'),
('US', 'United States of America', '840', 'North America'),
-- Latin America (27)
('AR', 'Argentina', '032', 'Latin America'),
('BS', 'Bahamas', '044', 'Latin America'),
('BZ', 'Belize', '084', 'Latin America'),
('BO', 'Bolivia', '068', 'Latin America'),
('BR', 'Brazil', '076', 'Latin America'),
('CL', 'Chile', '152', 'Latin America'),
('CO', 'Colombia', '170', 'Latin America'),
('CR', 'Costa Rica', '188', 'Latin America'),
('CU', 'Cuba', '192', 'Latin America'),
('DO', 'Dominican Rep.', '214', 'Latin America'),
('EC', 'Ecuador', '218', 'Latin America'),
('SV', 'El Salvador', '222', 'Latin America'),

```

```

('FK', 'Falkland Is.', '238', 'Latin America'),
('GT', 'Guatemala', '320', 'Latin America'),
('GY', 'Guyana', '328', 'Latin America'),
('HT', 'Haiti', '332', 'Latin America'),
('HN', 'Honduras', '340', 'Latin America'),
('JM', 'Jamaica', '388', 'Latin America'),
('NI', 'Nicaragua', '558', 'Latin America'),
('PA', 'Panama', '591', 'Latin America'),
('PY', 'Paraguay', '600', 'Latin America'),
('PE', 'Peru', '604', 'Latin America'),
('PR', 'Puerto Rico', '630', 'Latin America'),
('SR', 'Suriname', '740', 'Latin America'),
('TT', 'Trinidad and Tobago', '780', 'Latin America'),
('UY', 'Uruguay', '858', 'Latin America'),
('VE', 'Venezuela', '862', 'Latin America'),
-- Africa & Oceania (57)
('DZ', 'Algeria', '012', 'Africa & Oceania'),
('AO', 'Angola', '024', 'Africa & Oceania'),
('AU', 'Australia', '036', 'Africa & Oceania'),
('BJ', 'Benin', '204', 'Africa & Oceania'),
('BW', 'Botswana', '072', 'Africa & Oceania'),
('BF', 'Burkina Faso', '854', 'Africa & Oceania'),
('BI', 'Burundi', '108', 'Africa & Oceania'),
('CM', 'Cameroon', '120', 'Africa & Oceania'),
('CF', 'Central African Rep.', '140', 'Africa & Oceania'),
('TD', 'Chad', '148', 'Africa & Oceania'),
('CG', 'Congo', '178', 'Africa & Oceania'),
('CI', 'Côte d'Ivoire', '384', 'Africa & Oceania'),
('CD', 'Dem. Rep. Congo', '180', 'Africa & Oceania'),
('DJ', 'Djibouti', '262', 'Africa & Oceania'),
('GQ', 'Eq. Guinea', '226', 'Africa & Oceania'),
('ER', 'Eritrea', '232', 'Africa & Oceania'),
('ET', 'Ethiopia', '231', 'Africa & Oceania'),
('FJ', 'Fiji', '242', 'Africa & Oceania'),
('TF', 'Fr. S. Antarctic Lands', '260', 'Africa & Oceania'),
('GA', 'Gabon', '266', 'Africa & Oceania'),
('GM', 'Gambia', '270', 'Africa & Oceania'),
('GH', 'Ghana', '288', 'Africa & Oceania'),
('GN', 'Guinea', '324', 'Africa & Oceania'),
('GW', 'Guinea-Bissau', '624', 'Africa & Oceania'),
('KE', 'Kenya', '404', 'Africa & Oceania'),
('LS', 'Lesotho', '426', 'Africa & Oceania'),
('LR', 'Liberia', '430', 'Africa & Oceania'),
('LY', 'Libya', '434', 'Africa & Oceania'),
('MG', 'Madagascar', '450', 'Africa & Oceania'),
('MW', 'Malawi', '454', 'Africa & Oceania'),
('ML', 'Mali', '466', 'Africa & Oceania'),
('MR', 'Mauritania', '478', 'Africa & Oceania'),
('MA', 'Morocco', '504', 'Africa & Oceania'),
('MZ', 'Mozambique', '508', 'Africa & Oceania'),
('NA', 'Namibia', '516', 'Africa & Oceania'),
('NC', 'New Caledonia', '540', 'Africa & Oceania'),
('NZ', 'New Zealand', '554', 'Africa & Oceania'),
('NE', 'Niger', '562', 'Africa & Oceania'),
('NG', 'Nigeria', '566', 'Africa & Oceania'),
('PG', 'Papua New Guinea', '598', 'Africa & Oceania'),
('RW', 'Rwanda', '646', 'Africa & Oceania'),
('SS', 'S. Sudan', '728', 'Africa & Oceania'),
('SN', 'Senegal', '686', 'Africa & Oceania'),
('SL', 'Sierra Leone', '694', 'Africa & Oceania'),
('SB', 'Solomon Is.', '090', 'Africa & Oceania'),
('SO', 'Somalia', '706', 'Africa & Oceania'),
('ZA', 'South Africa', '710', 'Africa & Oceania'),
('SD', 'Sudan', '729', 'Africa & Oceania'),

```

```

('TZ', 'Tanzania', '834', 'Africa & Oceania'),
('TG', 'Togo', '768', 'Africa & Oceania'),
('TN', 'Tunisia', '788', 'Africa & Oceania'),
('UG', 'Uganda', '800', 'Africa & Oceania'),
('VU', 'Vanuatu', '548', 'Africa & Oceania'),
('EH', 'W. Sahara', '732', 'Africa & Oceania'),
('ZM', 'Zambia', '894', 'Africa & Oceania'),
('ZW', 'Zimbabwe', '716', 'Africa & Oceania'),
('SZ', 'eSwatini', '748', 'Africa & Oceania')
) as v (country_code, name, map_id, region_name)
join regions r on r.name = v.region_name;

-- Check: the join above silently drops a row whose region name has no match,
-- so this should return 173:
-- select count(*) from countries;
```